

Problem Solving Using Python (CSE1012)

Dr. Abdul Kayom, Assistant Professor, SENSE

Agenda / Contents

- 01 Introduction to File Handling
- 02
- 03
- 04
- 05

Introduction to File handling

- File is a collection of data stored on a secondary storage like hard disk
- Till now we have been processing data that was entered through keyboard
- This task can become very tedious especially when huge amount of data to be processed

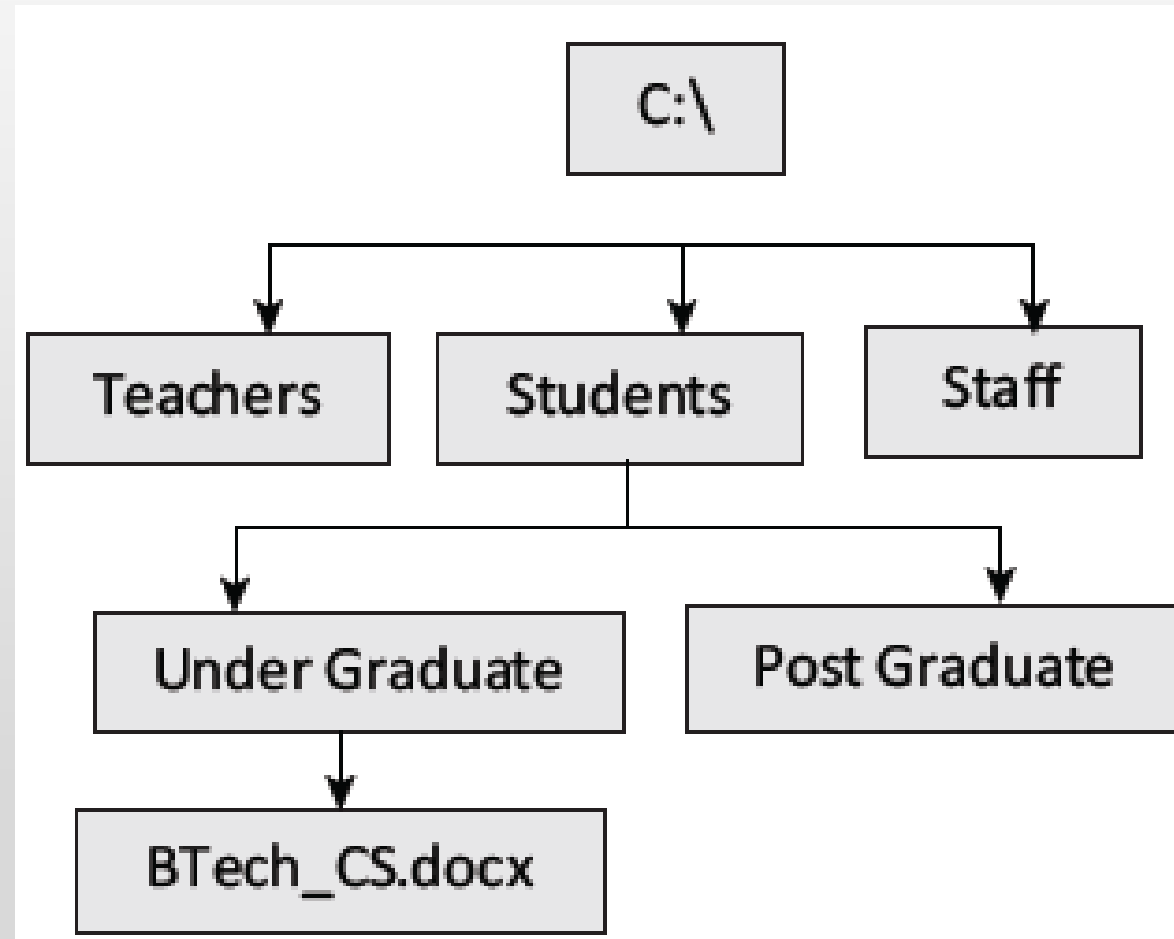
Introduction to File handling

- A better solution is to combine all the input data into a file and then design a Python program to read this data whenever required
- To work with a file data, you need to open it, then read/write data, and then finally close the file

File Path

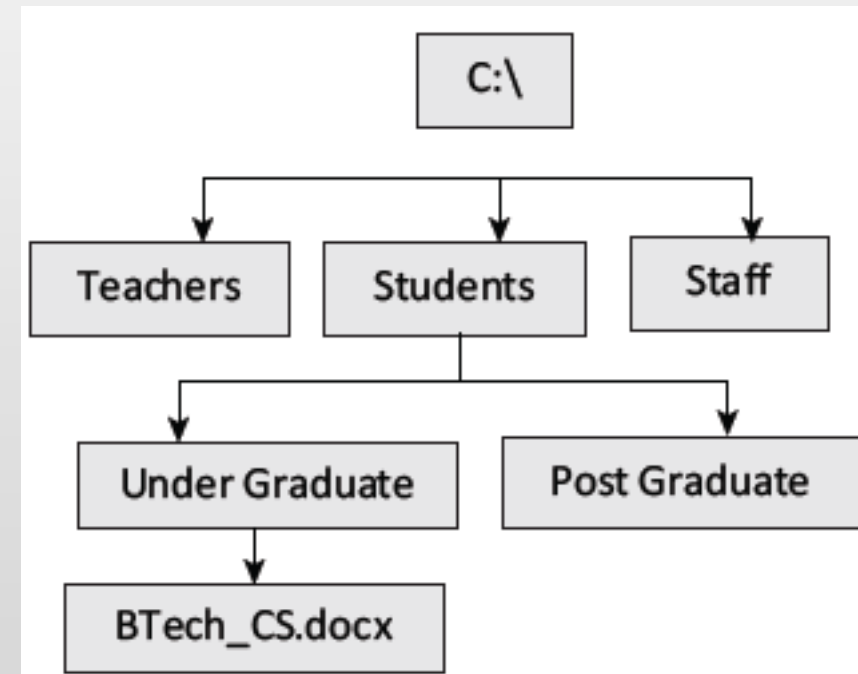
- Files are stored on a storage medium like hard disk
- Most file systems that are used stores files in a tree (or hierarchical) structure
- Every file is identified by its path that begins from the root node or the root folder

File Path



File Path

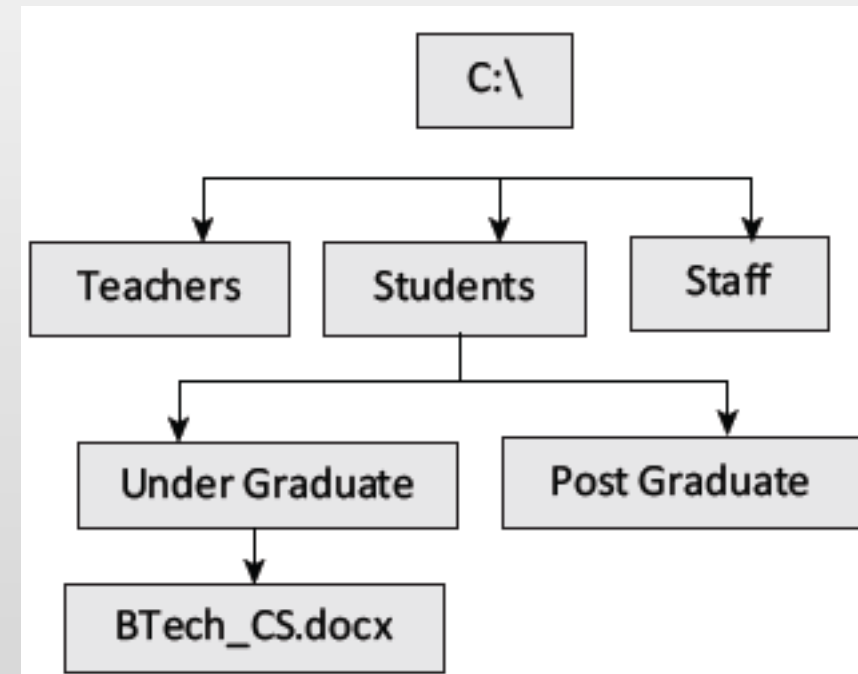
- The file Btech_CS.docx file is stored in the C:\.
- C-drive has a folder Students which in turn has a sub-folder named Under Graduate
- The sub-folder Under Graduate has the desired file



File Path

- The path of this file can be written as,

C:\Students\Under Graduate\BTech_CS.docx



Opening of Files

- Before reading from a file or writing into a file, you must first open it
- Open file using `open()` function
- It creates a file object, which will be used to invoke methods associated with it

```
fileobj = open(file_name , access_mode)
```

Opening of Files

- `file_name` is a string value that specifies name of the file that you want to access
- `access_mode` indicates in which mode the file has to be opened
- i.e. read, write, append, etc.

```
fileobj = open(file_name , access_mode)
```

Opening of Files `open()` Function

- The `open()` function returns a file object
- This file object will be used to read, write, or perform any other operation on the file
- `access_mode` is optional (default file access mode is read (r))

```
fileobj = open(file_name , access_mode)
```

Opening of Files open() Function

```
file = open("File1.txt", "rb")  
print(file)
```

OUTPUT

```
<open file 'File1.txt', mode 'rb' at 0x02A850D0>
```

rb: read only, opens file in binary format. The file pointer is placed at the beginning of the file.

Access Modes

r: It is read only mode. It is the default mode. File pointer is placed at the beginning of the file.

rb: read only, opens file in binary format. The file pointer is placed at the beginning of the file.

r+: opens file for both reading and writing. The file pointer is placed at the beginning of the file.

Access Modes

`rb+`: opens in both read write mode in binary format. File pointer is placed at the beginning of the file.

`w`: write only mode. If the file doesn't exist, a new file is created for writing. If file exists and has some data in it, the contents are overwritten.

`w+`: for both write and read.

Access Modes

`wb+`: opens file for both read write mode.

`a`: opens file for appending. File pointer is placed at the end of the file.

`ab`: opens in binary format for appending. File pointer is placed at the end of the file.

Access Modes

a+: opens file in both reading and appending. File pointer is placed at the end of the file.

ab+: opens file in binary format for both reading and appending. File pointer is placed at the end of the file.

Access Modes

Mode	Purpose
r	This is the default mode of opening a file which opens the file for reading only. The file pointer is placed at the beginning of the file.
rb	This mode opens a file for reading only in binary format. The file pointer is placed at the beginning of the file.

Access Modes

r+	This mode opens a file for both reading and writing. The file pointer is placed at the beginning of the file.
rb+	This mode opens the file for both reading and writing in binary format. The file pointer is placed at the beginning of the file.
w	This mode opens the file for writing only. When a file is opened in w mode, two things can happen. If the file does not exist, a new file is created for writing. If the file already exists and has some data stored in it, the contents are overwritten.
wb	Opens a file in binary format for writing only. When a file is opened in this mode, two things can happen. If the file does not exist, a new file is created for writing. If the file already exists and has some data stored in it, the contents are overwritten.
w+	Opens a file for both writing and reading. When a file is opened in this mode, two things can happen. If the file does not exist, a new file is created for reading as well as writing. If the file already exists and has some data stored in it, the contents are overwritten.
wb+	Opens a file in binary format for both reading and writing. When a file is opened in this mode, two things can happen. If the file does not exist, a new file is created for reading as well as writing. If the file already exists and has some data stored in it, the contents are overwritten.
a	Opens a file for appending. The file pointer is placed at the end of the file if the file exists. If the file does not exist, it creates a new file for writing.
ab	Opens a file in binary format for appending. The file pointer is at the end of the file if the file exists. If the file does not exist, it creates a new file for writing.
a+	Opens a file for both reading and appending. The file pointer is placed at the end of the file if the file exists. If the file does not exist, it creates a new file for reading and writing.
ab+	Opens a file in binary format for both reading and appending. The file pointer is placed at the end of the file if the file exists. If the file does not exist, a new file is created for reading and writing.

File Object Attributes

- Once a file is successfully opened, a file object is returned
- Using this file object, you can easily access different types of information related to that file
- This information can be obtained by reading values of specific attributes of the file

File Object Attributes

Attributes	Information Obtained
<code>fileobj.closed</code>	Returns True if the file is closed and False otherwise
<code>Fileobj.mode</code>	Returns access mode with which file has been opened
<code>Fileobj.name</code>	Returns name of the file

File Object Attributes

```
file = open("File1.txt", "wb")  
print("Name of the file: ", file.name)  
print("File is closed.", file.closed)  
  
print("File has been opened in ", file.mode, "mode")
```

OUTPUT

```
Name of the file:  File1.txt  
File is closed. False  
File has been opened in wb mode
```

Close() Method

- The close() method as the name suggests is used to close the file object
- Once a file object is closed, you can't further read or write into the file associated with the file object
- While closing the file object, the close() flushes any unwritten information (i.e. transfers the data that was supposed to be written in the file but has not been transferred)

Close() Method

- Python automatically closes a file when the reference object of a file is reassigned to another file
- But you should always explicitly use the close() method to close a file

`fileobj.close()` #The syntax of close() method

Close() Method

```
file = open("File1.txt", "wb")  
print("Name of the file: ", file.name)  
print("File is close: ", file.closed)  
print("The file is being closed...")  
file.close()  
print("File is closed: ", file.closed)  
print(file.read())    #This will show error, since the file is closed
```


Close() Method

```
file = open("File1.txt", "w")  
file.write("Hello All, hope you are enjoying learning Python")  
file.close()  
print("Data Written into the file.....")
```

OUTPUT

```
Data Written into the file....."
```

Reading & Writing Files

- The `read()` function is used to read data from a file
- The `write()` function is used to write data to files

write() Method

- The write() method is used to write a string to an already opened file
- This string may include numbers, special characters, or other symbols
- Remember that the write() method doesn't add a newline character ('\n') to the end of the string

`fileobj.write(string)`

write() Method

- The string that is passed as an argument to write(), is written into the opened file

`fileobj.write(string)`

write() Method

```
file = open("File1.txt", "w")
```

```
file.write("Hello everyone, hope you are enjoying learning  
Python")
```

```
file.close()
```

If you open the file, you should see the lines written in it.

write() Method

#If you open the file, you should see the lines written in it.

```
file = open("File1.txt",'r')
```

```
file.read()
```

```
'Hello everyone, I hope you are enjoying learning Python'
```

Thank You

Dr. Abdul Kayom



Asst. Prof, (SENSE)



kayomabdul@vitap.ac.in



8474860025